Лекция 8. Многозадачность, процессы, потоки и мониторинг ресурсов ОС

Цель лекции

- Понять назначение многозадачности в ОС.
- Различать процесс, поток и службу.
- Объяснять жизненный цикл процесса.
- Понимать кооперативную и вытесняющую многозадачность.
- Выполнять базовый мониторинг процессов и ресурсов в Linux и Windows.

Список учебных вопросов

1. Назначение многозадачности в операционной системе.
2. Одновременная работа приложений, служб и фоновых процессов.
3. Процесс как выполняемая программа: PID, ресурсы процесса, открытые файлы и сетевые соединения.
4. Потоки: назначение, отличие процесса от потока, примеры применения.
5. Виды многозадачности: кооперативная и вытесняющая.
6. Жизненный цикл процесса.
7. Процессы-сироты и zombie-процессы в Linux.
8. Управление процессами и службами в Linux и Windows.
9. Мониторинг ресурсов: top, htop, ps, free, Task Manager, Resource Monitor.

Основные термины лекции

- Multitasking — многозадачность, способность ОС выполнять несколько задач.
- Process — процесс, выполняемый экземпляр программы.
- PID — Process ID, идентификатор процесса.
- Thread — поток выполнения внутри процесса.
- Service — служба, фоновый процесс для системной или серверной функции.
- Signal — сигнал, способ управления процессами в Linux.

1. Назначение многозадачности

Многозадачность позволяет ОС выполнять несколько задач одновременно.

На практике CPU быстро переключается между процессами.

Пользователь видит одновременную работу приложений.

Сервер может обслуживать несколько клиентов.

Без многозадачности современная ОС была бы непригодна для администрирования.

Зачем это администратору

- Нужно понимать, почему система тормозит.
- Нужно находить процессы, которые потребляют ресурсы.
- Нужно отличать нормальную нагрузку от зависания.
- Нужно управлять службами и фоновыми задачами.
- Нужно понимать, что происходит на сервере без графического интерфейса.

2. Одновременная работа приложений и служб

На компьютере одновременно работают приложения пользователя и службы.

- Фоновые процессы выполняют системные задачи.
- Службы предоставляют сетевые и серверные функции.
- ОС распределяет CPU, RAM, диск и сеть между задачами.
- Конфликт ресурсов влияет на все приложения и службы.

Пример одновременной работы

- Пользователь открыл браузер и офисный документ.
- Антивирус выполняет проверку.
- Служба обновлений загружает пакеты.
- Web-сервер отвечает на запросы.
- ОС планирует выполнение всех этих задач.

3. Процесс как выполняемая программа

- Процесс — запущенный экземпляр программы.
- У процесса есть PID.
- У процесса есть собственное адресное пространство.
- Процесс потребляет CPU, RAM, disk и network.
- Процесс может запускаться пользователем, системой или службой.

Ресурсы процесса

- CPU time: процессорное время.
- RAM: память процесса.
- Открытые файлы.
- Сетевые соединения.
- Переменные окружения.
- Права пользователя, от имени которого процесс запущен.

PID

- PID — Process ID, числовой идентификатор процесса.
- PID нужен для поиска и управления процессом.
- Команды диагностики часто работают с PID.
- После завершения процесса PID может быть использован повторно.

4. Потоки

- Поток — единица выполнения внутри процесса.
- Один процесс может иметь один или несколько потоков.
- Потоки одного процесса разделяют часть ресурсов.
- Потоки помогают выполнять несколько действий параллельно.
- Ошибка в одном процессе может затронуть его потоки.

Процесс и поток

- Процесс имеет отдельное адресное пространство.
- Потоки внутри процесса используют общие ресурсы процесса.
- Процесс тяжелее создать, чем поток.
- Потоки удобны для параллельной обработки задач.
- Администратору важно понимать это при анализе нагрузки.

Примеры потоков

- Браузер обрабатывает вкладки и сетевые запросы.
- Web-сервер обслуживает несколько клиентов.
- Антивирус сканирует файлы и обновляет базы.
- Служба базы данных обрабатывает несколько запросов.
- Многопоточность повышает производительность при правильной нагрузке.

ПРОЦЕСС И ПОТОКИ

Основы многозадачности в операционных системах

Процесс – исполняемая программа в системе

Поток (thread) – поток управления внутри процесса

Многозадачность – одновременное выполнение многих задач

Параллелизм – выполнение задач на нескольких ядрах

ЧТО ТАКОЕ ПРОЦЕСС?

Процесс – это экземпляр программы в выполнении.
Имеет своё адресное пространство и системные ресурсы.

- Адресное пространство
- Код программы (text)
- Данные (data)
- Куча (heap)
- Стек (stack)
- Открытые файлы
- Дескрипторы и др. ресурсы

Состояния процесса

- New (создание)
- Ready (готов)
- Running (выполняется)
- Waiting / Blocked (ожидание)
- Terminated (завершён)

ЧТО ТАКОЕ ПОТОК?

Поток – это минимальная единица планирования внутри процесса.
Потоки одного процесса разделяют общее адресное пространство и ресурсы.



Каждый поток имеет:

- Собственный стек
- Счётчик команд (PC)
- Регистры процессора
- Состояние выполнения

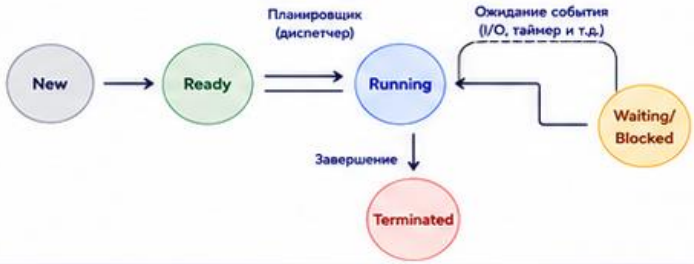
Преимущества потоков

- ✔ Лёгкое создание и переключение
- ✔ Быстрое взаимодействие
- ✔ Экономия памяти
- ✔ Эффективное использование ресурсов

ПРОЦЕССЫ vs ПОТОКИ

Критерий	Процесс	Поток
Адресное пространство	Отдельное	Общее в рамках процесса
Память	Больше (из-за изоляции)	Меньше (общая память)
Создание	Тяжёлое (медленнее)	Лёгкое (быстрее)
Переключение контекста	Дороже	Дешевле
Изоляция и безопасность	Высокая	Ниже (потоки одного процесса)
Отказ одного	Не влияет на другие	Может повлиять на все потоки процесса
Применение	Независимые задачи	Параллельные подзадачи внутри одной задачи

ЖИЗНЕННЫЙ ЦИКЛ ПРОЦЕССА



МНОГОПОТОЧНОСТЬ: МОДЕЛИ



- \+ Быстро в пользовательском режиме
- \- Блокировка одного потока блокирует все
- \+ Истинный параллелизм на многоядерных ЦП
- \+ Один заблокированный поток не блокирует другие

СОСТОЯНИЯ ПОТОКА



СОЗДАНИЕ ПРОЦЕССОВ

- Unix/Linux: `fork()`, `exec()`
- Windows: `CreateProcess()`
- Родительский процесс создаёт дочерний
- Дочерний процесс может выполнять другую программу

Копирование ресурсов:

- ✔ Unix (fork): копируется адресное пространство (Copy-on-Write)
- ✔ Windows: создаётся новое адресное пространство

СОЗДАНИЕ ПОТОКОВ

- Unix/Linux (POSIX): `pthread_create()`
- Windows: `CreateThread()`
- Поток существует только внутри процесса
- Завершение: `pthread_exit()` / `ExitThread()`
- Ожидание завершения: `pthread_join()` / `WaitForSingleObject()`

ПРИМЕНЕНИЕ ПОТОКОВ

- Параллельные вычисления
- Обработка ввода-вывода
- Серверы (много клиентов)
- Графические интерфейсы
- Фоновые задачи
- Повышение отзывчивости приложений

ВАЖНО ЗНАТЬ

- Процесс – тяжёлая структура, потоки – лёгкие.
- Потоки эффективны для разделения работы внутри одной задачи.

СИНХРОНИЗАЦИЯ ПОТОКОВ

Потоки одного процесса разделяют память, поэтому необходимы механизмы синхронизации:

- Мьютексы (mutex)
- Семафоры (semaphore)
- Условные переменные (condition variables)

ПРОБЛЕМЫ

- Гонки данных (race conditions)
- Взаимные блокировки (deadlocks)
- Сложность отладки многопоточных программ

5. Виды многозадачности

- Кооперативная многозадачность зависит от поведения программ.
- Вытесняющая многозадачность управляется ОС.
- Современные ОС используют вытесняющую многозадачность.
- Планировщик ОС распределяет процессорное время.
- Это повышает устойчивость системы.

Кооперативная многозадачность

- Программа сама должна уступить управление.
- Если программа зависла, она может мешать другим задачам.
- Такой подход проще, но менее надежен.
- Исторически встречался в старых системах.
- Для современных серверов не подходит как основной механизм.

Вытесняющая многозадачность

- ОС сама решает, когда переключать задачи.
- Планировщик распределяет процессорное время.
- Зависшая программа не должна остановить всю ОС.
- Позволяет надежнее обслуживать много процессов.
- Используется в Linux и Windows.

6. Жизненный цикл процесса

- Создание процесса.
- Готовность к выполнению.
- Выполнение на CPU.
- Ожидание события или ввода-вывода.
- Завершение процесса.
- Освобождение ресурсов.

Состояния процесса

- Running: процесс выполняется.
- Ready: процесс готов, но ждет CPU.
- Sleeping или waiting: процесс ждет событие.
- Stopped: процесс остановлен.
- Zombie: процесс завершен, но запись о нем еще не забрана родителем.

ЖИЗНЕННЫЙ ЦИКЛ ПРОЦЕССА

Как процесс создаётся, выполняется и завершается в операционной системе

Процесс — экземпляр программы выполняющийся в системе

Состояние процесса меняется в течение его жизни

Планировщик ОС управляет выполнением процессов

Переходы зависят от событий и ресурсов системы

ОСНОВНЫЕ СОСТОЯНИЯ ПРОЦЕССА

New (Новый)
Процесс создаётся. ОС выделяет ресурсы и создаёт управляющий блок (PCB).

Ready (Готовый)
Процесс готов к выполнению и ожидает предоставления CPU.

Running (Выполняется)
Процесс выполняется на процессоре.

Waiting / Blocked (Ожидание)
Процесс ожидает события или освобождения ресурса (ввод-вывод, память и т.д.).

Terminated (Завершён)
Процесс завершил выполнение. Ресурсы освобождаются ОС.

КАК ПРОИСХОДЯТ ПЕРЕХОДЫ

- Создание процесса
- Допуск к выполнению (выбор планировщиком)
- Прерывание / вытеснение (таймер, более приоритетный процесс и т.д.)
- Ожидание события или ресурса (I/O, семафор, страничный сбой и т.д.)
- Событие произошло / ресурс освободился
- Завершение (нормальное или по ошибке)

ДИАГРАММА СОСТОЯНИЙ



ОПИСАНИЕ СОСТОЯНИЙ

Состояние	Краткое описание	Где находится процесс
New	Процесс создаётся	На этапе создания (польз. процесс или системный вызов fork/exec)
Ready	Готов к выполнению, ждёт CPU	В очереди готовых процессов (Ready Queue)
Running	Использует CPU	На процессоре
Waiting / Blocked	Ожидает событие или ресурс	В очереди ожидания события (Wait Queue)
Terminated	Завершился, ресурсы освобождены	Нет в очередях, информация в журнале ОС

ПРИМЕРЫ СОБЫТИЙ, ВЫЗЫВАЮЩИХ ПЕРЕХОДЫ

- Запрос ввода-вывода (чтение файла, сетевая операция)
- Ожидание семафора / мьютекса
- Страничный сбой (page fault)
- Получение сигнала (signal)
- Завершение работы (exit), ошибка (kill)
- Истечение кванта времени (таймер)

ПРИМЕР ЖИЗНЕННОГО ЦИКЛА



ЖИЗНЕННЫЙ ЦИКЛ В КОДЕ (ПРИМЕР)

```
1  pid = fork();                  // Создание процесса
2  if (pid == 0) {                // Дочерний процесс
3     execlp("program", "program", NULL); // Выполнение программы
4  } else if (pid > 0) {          // Родительский процесс
5     wait(NULL);                 // Ожидание завершения дочернего
6  }
```

ПОЛЕЗНЫЕ КОМАНДЫ (LINUX)

`ps aux`	Показать запущенные процессы
`top / htop`	Мониторинг процессов в реальном времени
`pstree -p`	Дерево процессов
`kill <PID>`	Завершить процесс по PID
`kill -9 <PID>`	Принудительно завершить процесс
`nice -n <N> <cmd>`	Запустить процесс с приоритетом
`renice <N> -p <PID>`	Изменить приоритет уже запущенного процесса

ВАЖНО ЗНАТЬ Жизненный цикл процесса — основа многозадачности ОС. Эффективное управление состояниями процессов позволяет системе рационально распределять ресурсы и обеспечивать стабильную и быструю работу приложений.

7. Процессы-сироты и zombie-процессы

- Процесс-сирота — процесс, родитель которого завершился.
- ОС назначает такому процессу нового родителя.
- Zombie - процесс уже завершился, но запись о нем остается.
- Zombie обычно не потребляет CPU, но занимает запись в таблице процессов.
- Большое количество zombie-процессов указывает на проблему родительской программы.

Почему zombie важен администратору

- Один zombie-процесс не всегда критичен.
- Массовые zombie могут указывать на ошибку службы.
- Нужно искать родительский процесс.
- Простое kill для zombie обычно не решает проблему.
- Часто требуется перезапуск или исправление родительской службы.

8. Управление процессами и службами

- Администратор должен уметь найти процесс.
- Должен понимать, какой пользователь запустил процесс.
- Должен видеть потребление CPU и RAM.
- Должен уметь корректно завершить процесс.
- Должен проверять состояние служб.

Linux: управление процессами

- `ps aux` показывает процессы.
- `top` и `htop` показывают нагрузку.
- `pgrep` ищет PID по имени.
- `kill` отправляет сигнал процессу.
- `systemctl` управляет службами systemd.

Windows: управление процессами

- Task Manager показывает процессы и нагрузку.
- Details показывает PID.
- Services показывает службы.
- Resource Monitor показывает ресурсы подробнее.
- PowerShell позволяет автоматизировать просмотр процессов.

9. Мониторинг ресурсов

- Мониторинг помогает понять причину замедления.
- Нужно смотреть CPU, RAM, disk и network вместе.
- Один высокий показатель не всегда означает ошибку.
- Важно понимать нормальную нагрузку системы.
- Измерение лучше предположений.

Linux-инструменты мониторинга

- `top`: процессы и нагрузка.
- `htop`: интерактивный просмотр.
- `ps`: снимок списка процессов.
- `free`: память.
- `df` и `iostat`: диск и ввод-вывод при наличии.

Windows-инструменты мониторинга

- Task Manager: быстрый обзор.
- Resource Monitor: память, диск, сеть, CPU.
- Event Viewer: ошибки и события.
- Performance Monitor: длительный сбор счетчиков.
- PowerShell: автоматизация диагностики.

Итоги лекции

- Многозадачность позволяет ОС выполнять много задач.
- Процесс имеет PID, ресурсы и состояние.
- Потоки выполняются внутри процесса.
- Службы работают в фоне и обеспечивают системные функции.
- Администратор должен уметь находить перегрузку и управлять процессами.

Контрольные вопросы

1. Для чего операционной системе нужна многозадачность?
2. Что такое процесс и какие ресурсы он использует?
3. Чем поток отличается от процесса?
4. Почему современные ОС используют вытесняющую многозадачность?
5. Какие этапы входят в жизненный цикл процесса?
6. Что такое zombie-процесс на базовом уровне?
7. Какими средствами можно посмотреть процессы в Linux и Windows?
8. Как определить процесс, который потребляет слишком много CPU или RAM?
9. Почему администратору важно различать процесс, поток и службу?